

Patrón de Diseño Strategy

1. Problema

En algunos sistemas es necesario realizar una determinada tarea utilizando diferentes algoritmos o formas de resolver un mismo problema.

Una solución tradicional consiste en colocar todas las alternativas dentro de una misma clase utilizando estructuras condicionales como `if`, `else` o `switch`. Sin embargo, cuando la cantidad de alternativas aumenta, el código puede volverse difícil de mantener, modificar y ampliar.

Por ejemplo, un sistema puede necesitar realizar diferentes operaciones matemáticas, como sumar, restar o multiplicar. Una implementación podría utilizar una única clase con diferentes condiciones para decidir qué operación realizar.

El problema de este enfoque es que cada vez que se incorpora una nueva operación es necesario modificar la clase existente. Esto puede aumentar la complejidad y dificultar el mantenimiento del programa.

Por lo tanto, el problema que busca resolver Strategy es **permitir utilizar diferentes algoritmos o comportamientos para resolver una misma tarea, evitando que todos ellos estén concentrados dentro de una única clase**.

Situaciones en las que puede utilizarse

Algunos ejemplos donde puede resultar útil son:

- Diferentes métodos de pago.
- Diferentes algoritmos de ordenamiento.
- Diferentes métodos de cálculo.
- Diferentes formas de aplicar descuentos.
- Diferentes algoritmos de navegación.
- Diferentes métodos de compresión o procesamiento de información.

2. Solución

El patrón Strategy propone encapsular cada algoritmo o comportamiento en una clase independiente.

Todas las estrategias comparten una interfaz común, lo que permite que el objeto que las utiliza pueda trabajar con cualquiera de ellas sin conocer los detalles de su implementación.

En el diagrama UML desarrollado para este proyecto, el patrón está compuesto por:

- La interfaz **Estrategia**.
- La clase **Contexto**.
- **EstrategiaSuma**.
- **EstrategiaResta**.
- **EstrategiaMultiplicacion**.

La interfaz **Estrategia** define el método **ejecutar()**, que representa la operación que deberán realizar las diferentes estrategias.

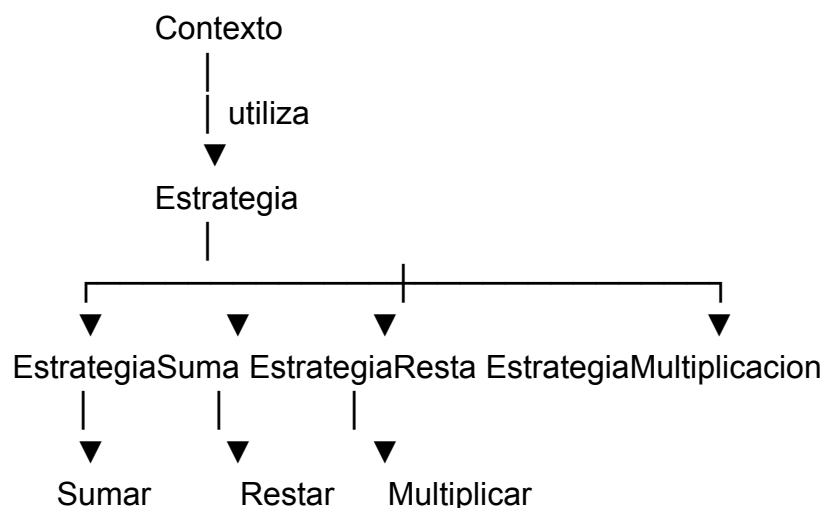
Las clases **EstrategiaSuma**, **EstrategiaResta** y **EstrategiaMultiplicacion** implementan esa interfaz y proporcionan una versión diferente del algoritmo.

La clase **Contexto** mantiene una referencia a una estrategia y delega en ella la ejecución de la operación. Además, permite cambiar la estrategia mediante el método **establecerEstrategia()**.

El funcionamiento puede resumirse de la siguiente manera:

1. Se define una interfaz común llamada **Estrategia**.
2. Se crean diferentes clases que implementan esa interfaz.
3. El **Contexto** mantiene una referencia a una estrategia.
4. El programa selecciona la estrategia que desea utilizar.
5. El **Contexto** delega la operación a la estrategia seleccionada.
6. La estrategia ejecuta su propio algoritmo.
7. La estrategia puede cambiarse durante la ejecución del programa.

Conceptualmente:



Por ejemplo, inicialmente el `Contexto` puede utilizar `EstrategiaSuma`. Posteriormente, mediante `establecerEstrategia()`, puede cambiar a `EstrategiaResta` sin necesidad de modificar la clase `Contexto`.

De esta manera, cada algoritmo se encuentra separado en su propia clase y el código resulta más organizado y fácil de ampliar.

3. Consecuencias

La utilización del patrón Strategy presenta ventajas y desventajas que deben ser consideradas antes de aplicarlo.

- **Ventajas:**

Permite intercambiar algoritmos

Una de las principales ventajas es que permite cambiar el comportamiento utilizado por el programa durante la ejecución.

Reduce las estructuras condicionales

Permite evitar grandes bloques de `if`, `else` o `switch` utilizados para seleccionar diferentes algoritmos.

Facilita la incorporación de nuevas estrategias

Para agregar un nuevo algoritmo, normalmente se puede crear una nueva clase que implemente la interfaz `Estrategia`, sin modificar las estrategias existentes.

Favorece la separación de responsabilidades

Cada estrategia se encarga de un comportamiento específico, evitando concentrar diferentes algoritmos en una sola clase.

Facilita las pruebas

Cada estrategia puede probarse de manera independiente, lo que permite comprobar cada algoritmo sin depender de los demás.

- **Desventajas:**

Aumenta la cantidad de clases

Cada algoritmo se representa mediante una clase diferente. En sistemas con muchos comportamientos, esto puede aumentar considerablemente la cantidad de clases.

El cliente debe conocer las estrategias

En muchos casos, el código que utiliza el patrón necesita conocer las diferentes estrategias disponibles para poder seleccionar cuál utilizar.

Puede aumentar la complejidad para problemas simples

Si solamente existe un algoritmo o las alternativas son muy pequeñas, utilizar Strategy puede agregar una estructura innecesaria al programa.

Las estrategias deben mantener una interfaz común

Los diferentes algoritmos deben poder adaptarse a la interfaz definida por **Estrategia**, lo que puede resultar poco conveniente si sus necesidades son muy diferentes.

- **Implicancias:**

Strategy resulta especialmente útil cuando un sistema posee diferentes algoritmos para realizar una misma tarea y se espera que estas alternativas puedan cambiar o crecer con el tiempo.

Una de las principales implicancias del patrón es que separa el algoritmo del objeto que lo utiliza. Esto permite modificar o incorporar nuevos comportamientos con menor impacto sobre el resto del sistema.

Sin embargo, su utilización debe justificarse según la complejidad del problema. Para una aplicación muy sencilla, crear una interfaz y varias clases puede ser innecesario. En cambio, cuando existen múltiples algoritmos o comportamientos que pueden variar, Strategy proporciona una estructura flexible y fácil de mantener.

En conclusión, el patrón Strategy permite **encapsular algoritmos intercambiables**, favoreciendo la flexibilidad, la extensibilidad y la separación de responsabilidades dentro del software.